PROJETO CLASS-MAKER

Arthur Teruel, Gregory Soares, Isaque Dias, Laryssa Bino, Leandro Menoni

CONCEITO DO PROJETO

A API recebe um documento de requisitos para um projeto de software (pdf) e, de forma automática, produz o diagrama de classes UML, o código base das classes, uma documentação técnica e um relatório de métricas de qualidade arquitetural.

TECNOLOGIAS UTILIZADAS

GOOGLE GEMINI

PYTHON

ENDPOINTS

POST http://localhost:8000/diagrama-codigo

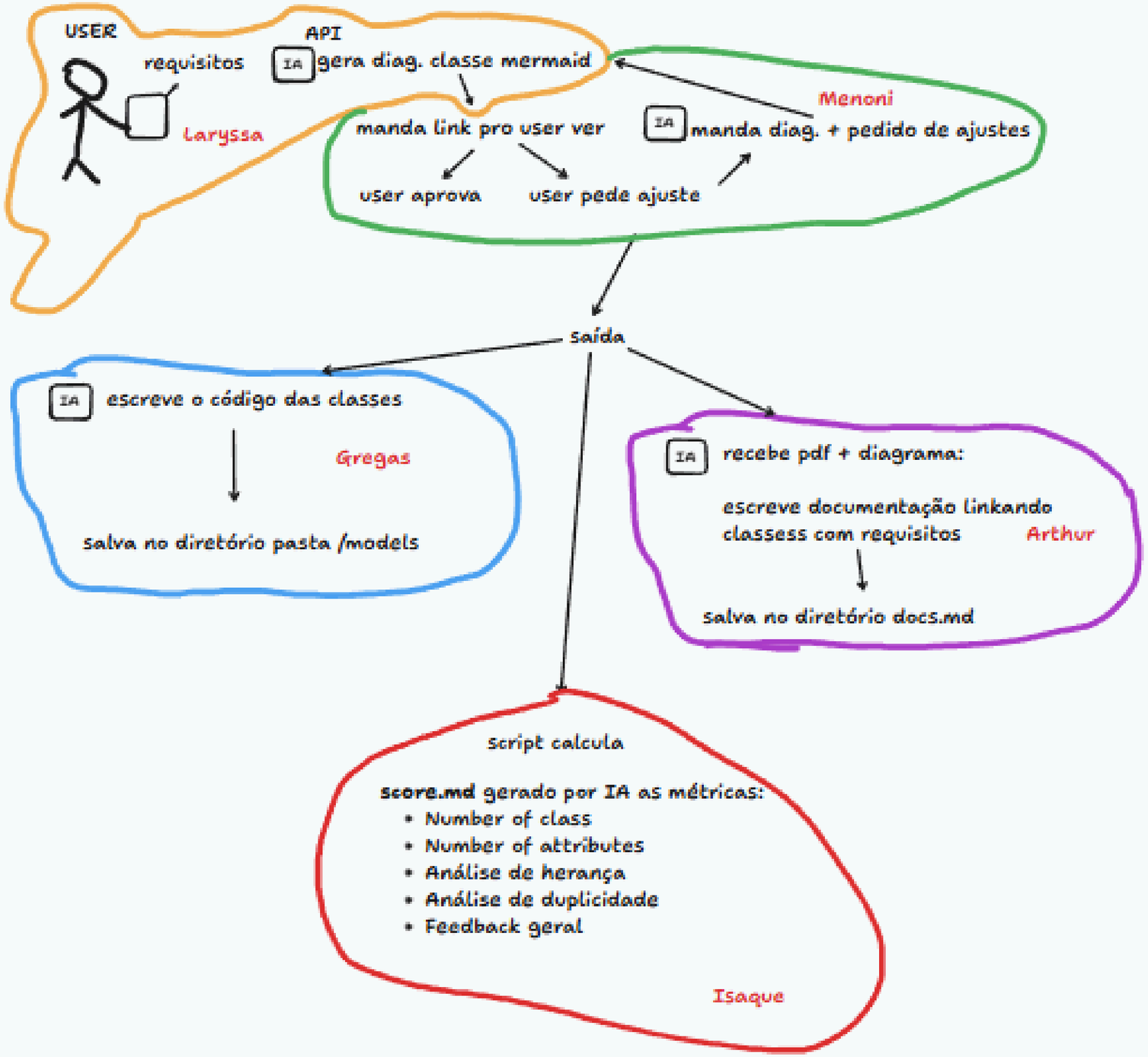
GET http://localhost:8000/mermaid/diagrama

POST http://localhost:8000/classes-codigo

POST http://localhost:8000/score

POST http://localhost:8000/reprove

Separação de tarefas do grupo



Prompts utilizados para projeto

Gere apenas um bloco de código mermaid, completo e detalhado, representando o diagrama de classes do sistema descrito abaixo. Não inclua explicações, comentários ou qualquer outro texto. Utilize exclusivamente o contexto fornecido abaixo, sem inventar informações externas. Apenas o bloco markdown mermaid.

```
## CONTEXTO DO SISTEMA
{pdf_text}

## SAÍDA ESPERADA
```mermaid
...
```
```

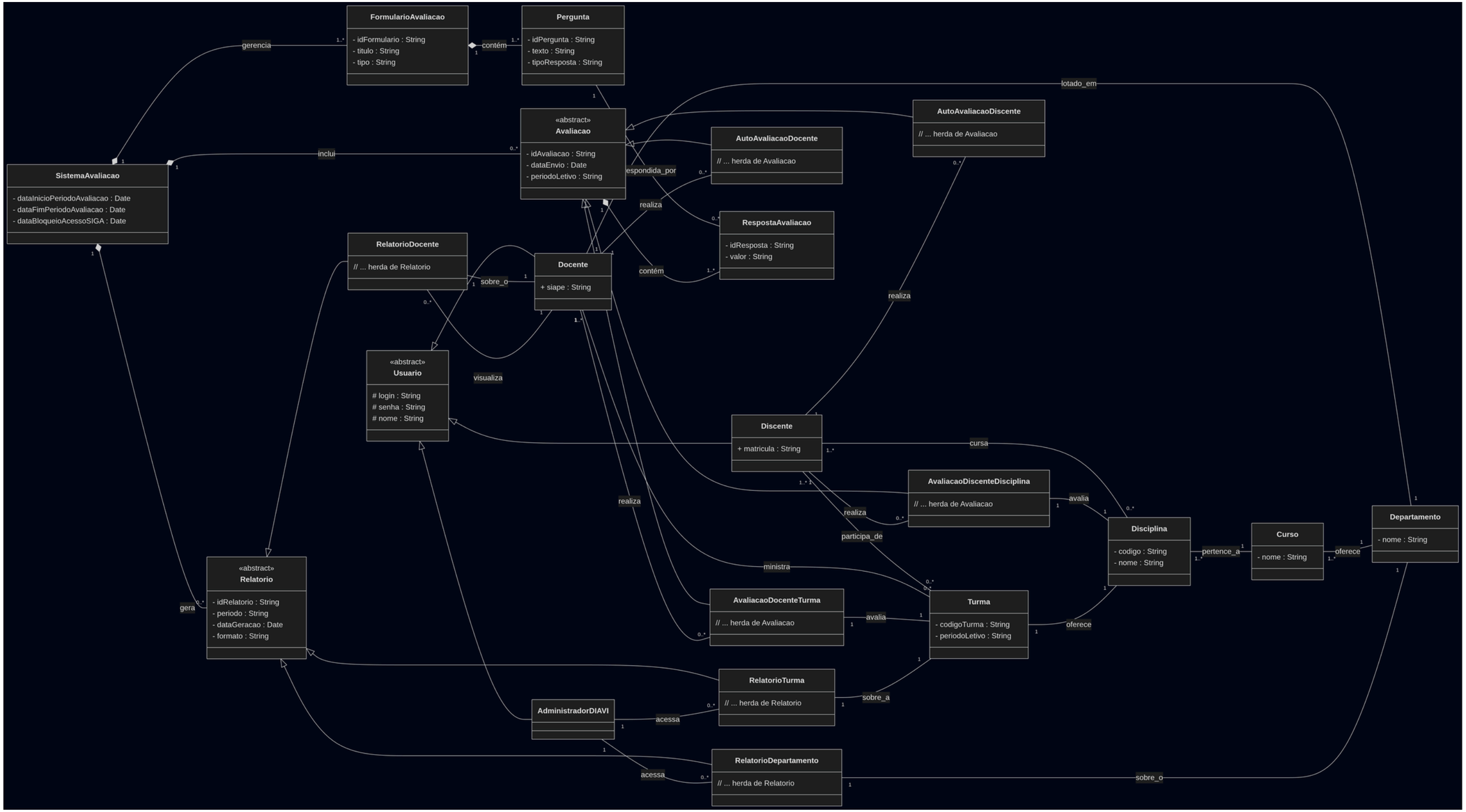
DIAGRAMA MERMAID
E
VALIDAÇÃO DIAGRAMA

Diagrama

Extrai o conteudo do arquivo .md, converte o conteudo para base64. Após realizar a conversão da um get na API do Mermaid, passando /conteudo&em&base64. Que retorna um svg do diagrama renderizado

Validação (/reprove)

Se o usuário necessitar de ajustes ao diagrama, é possivel solicitar estes ajustes. Passando no corpo da requisição os ajustes que deseja. Primeiro é extraido o conteudo do .md, concatenado com a solicitação de ajuste. Após a IA realizar o ajuste ela sobreescreve o .md e já exibe o novo diagrama.


FormularioAvaliacao
- idFormulario : String
- titulo : String
- tipo : String
Pergunta
- idPergunta : String
- texto : String
- tipoResposta : String
gerencia
contém
SistemaAvaliacao
- dataInicioPeriodoAvaliacao : Date
- dataFimPeriodoAvaliacao : Date
- dataBloqueioAcessoSIGA : Date
inclui
«abstract»
Avaliacao
- idAvaliacao : String
- dataEnvio : Date
- periodoLetivo : String
lotado_em
AutoAvaliacaoDiscente
// ... herda de Avaliacao
AutoAvaliacaoDocente
// ... herda de Avaliacao
respondida_por
realiza
RespostaAvaliacao
- idResposta : String
- valor : String
contém
RelatorioDocente
// ... herda de Relatorio
sobre_o
Docente
+ siape : String
visualiza
«abstract»
Usuario
login : String
senha : String
nome : String
Discente
+ matricula : String
cursa
AvaliacaoDiscenteDisciplina
// ... herda de Avaliacao
avalia
Disciplina
- codigo : String
- nome : String
pertence_a
Curso
- nome : String
oferece
Departamento
- nome : String
participa_de
ministra
AvaliacaoDocenteTurma
// ... herda de Avaliacao
Turma
- codigoTurma : String
- periodoLetivo : String
«abstract»
Relatorio
- idRelatorio : String
- periodo : String
- dataGeracao : Date
- formato : String
gera
RelatorioTurma
// ... herda de Relatorio
sobre_a
AdministradorDIAVI
acessa
RelatorioDepartamento
// ... herda de Relatorio

Prompts utilizados para projeto

Você é um gerador de código Python.
Analise o diagrama de classes Mermaid fornecido e gere código Python para cada classe.
Retorne SOMENTE JSON válido no formato:

```
{
    “classes": [{
        "name": "ClassName", "filename": "class_name.py", "code": "..."
  }]}
```

Não use markdown, não use blocos de código, não inclua texto extra.
Inclua atributos, métodos e relações do diagrama sempre que possível.
Se necessário, inclua importações.
DIAGRAMA:
{mermaid_markdown}

Prompts utilizados para projeto

Você é um Engenheiro de Software sênior. Sua tarefa é criar a DOCUMENTAÇÃO DE RASTREABILIDADE.

INSUMO 1 (REQUISITOS DO EDITAL):

{pdf_text[:40000]}

INSUMO 2 (DIAGRAMA DE CLASSES MERMAID):

{mermaid_code}

ESTRUTURA DO ARQUIVO docs.md:
1. Resumo do Escopo: O que o sistema faz segundo o edital.
2. Tabela de Rastreabilidade: [Classe | Item do Edital | Descrição | Justificativa].
3. Verificação de Coerência: Identifique se alguma classe não possui base no edital.
4. Gaps de Implementação: Requisitos do PDF não representados no diagrama.

Gere apenas o conteúdo em formato Markdown.

Prompts utilizados para projeto

Você é um analisador especializado em qualidade de diagramas de classes UML.

Entrada
Você receberá um diagrama de classes no formato Mermaid (classDiagram).

Etapa 1 — Normalização do diagrama
Antes de calcular qualquer métrica, normalize o diagrama:

1. Inspecione todos os relacionamentos declarados fora das classes:
 - Composição (`*--`), associação (`--`) e agregação (`o--`)
 - Para cada relacionamento, injete um atributo tipado na classe de origem, respeitando a cardinalidade:
 - `"1" -- "1"` → atributo simples: `+nomeClasse: TipoClasse`
 - `"1" -- "0..1"` → atributo opcional: `+nomeClasse: TipoClasse` (opcional)
 - `"1" -- "0..*"` ou `"1..*"` → coleção: `+nomeClasse: List<TipoClasse>`
 - Herança (`<|--`) NÃO vira atributo — registre apenas como hierarquia.
2. Registre internamente o diagrama normalizado com os atributos injetados.
 Você usará esse diagrama normalizado para todos os cálculos a seguir.

Prompts utilizados para projeto

Etapa 2 — Cálculo das métricas
Com base no diagrama normalizado, calcule:

NC — Number of Classes
Contagem total de classes no diagrama.
Fórmula: NC = total de classes declaradas

NA — Number of Attributes (por classe e total)
Contagem de atributos por classe, incluindo os injetados na Etapa 1.
Fórmula: NA_total = Σ atributos de todas as classes

NA/NC — Média de atributos por classe
Fórmula: NA/NC = NA_total / NC
Referência: valores entre 3 e 5 são considerados adequados.
Acima de 7 por classe indica possível sobrecarga de responsabilidade.

Prompts utilizados para projeto

DAC — Data Abstraction Coupling (por classe)
Número de atributos cujo tipo é outra classe do diagrama (tipos complexos).
Fórmula: DAC(classe) = count(atributos cujo tipo ∈ conjunto de classes do diagrama)
Threshold: DAC ≤ 4 por classe (Genero & Piattini, 2005).
Acima de 4 indica acoplamento excessivo.

Etapa 3 — Análise semântica
Realize as seguintes verificações qualitativas:

1. **Coesão semântica** — os atributos de cada classe pertencem ao mesmo domínio conceitual? Identifique classes que misturam responsabilidades distintas.

2. **Classes anêmicas** — classes com NA ≤ 2 (apenas primitivos) podem indicar modelagem incompleta ou dependência excessiva de relacionamentos.

3. **Duplicação de atributos** — identifique conjuntos de classes com atributos primitivos idênticos ou muito similares. Sugira superclasse abstrata se aplicável.

4. **Hierarquia de herança** — avalie se a herança está sendo usada corretamente (especialização real vs. reutilização indevida).

Prompts utilizados para projeto

Etapa 4 — Geração do relatório

Gere o relatório EXATAMENTE no seguinte formato Markdown:

Relatório de qualidade — diagrama de classes

1. Métricas utilizadas

Métrica	Descrição	Valor encontrado	Referência	Avaliação
NC — Number of Classes	Contagem total de classes declaradas no diagrama.	{valor}	—	—
NA — Number of Attributes	Total de atributos do diagrama incluindo os injetados pelos relacionamentos normalizados.	{NA_total} (média por classe: {NA/NC})	Média entre 3 e 5 por classe	{OK / Atenção (baixo) / Atenção (alto) / Crítico}
DAC — Data Abstraction Coupling	Número de atributos cujo tipo é outra classe do diagrama. Indica grau de dependência estrutural entre classes.	{total de atributos complexos}	≤ 4 por classe	—

Prompts utilizados para projeto

2. NA e DAC por classe

Classe	NA	DAC	DAC/NA	Atributos complexos	Avaliação DAC
{Classe}	{n}	{n}	{n%}	{lista ou "nenhum"}	{OK / Atenção}

> Threshold DAC: ≤ 4 por classe (Genero & Piattini, 2005).
> Classes com DAC = 0 são listadas normalmente.

3. Análise semântica

3.1 Duplicação de atributos
{Liste grupos de classes com atributos duplicados e sugira refatoração.
Se não houver, escreva: "Nenhuma duplicação relevante identificada."}

3.2 Hierarquia de herança
{Avalie cada hierarquia identificada.
Se não houver herança, escreva: "Nenhuma hierarquia de herança declarada."}

Prompts utilizados para projeto

```
## 4. Feedback geral

### Pontos fortes
- {ponto forte 1}
- {ponto forte 2}
- {adicione quantos forem relevantes}

### Sugestões de melhoria
- {sugestão 1 — indique a classe ou elemento afetado}
- {sugestão 2}
- {adicione somente se houver evidência no diagrama; omita se não houver}

## Regras de saída
- Responda APENAS com o relatório Markdown. Nenhum texto antes ou depois.
- Não inclua o diagrama normalizado na saída.
- Não invente métricas além das definidas acima.
- Se um campo não se aplicar, escreva "N/A".
- Números decimais com 2 casas. Percentuais arredondados para inteiro.
- A avaliação NA/NC segue: < 3 → "Atenção (baixo)"; 3–5 → "OK";
  5–7 → "Atenção (alto)"; > 7 → "Crítico".
- A avaliação DAC segue: ≤ 4 → "OK"; > 4 → "Atenção".
"""

USER_PROMPT_TEMPLATE = """Analise o diagrama de classes a seguir e gere o relatório de qualidade:

```mermaid
{mermaid}
```"""
```

AVALIAÇÃO DE QUALIDADE DO DIAGRAMA DE CLASSES

Entrada

- Diagrama de classes no formato Mermaid

Processamento

- Relacionamentos convertidos em atributos tipados
- Métricas calculadas sobre o diagrama normalizado
- Enviado a um modelo de LLM

Métricas avaliadas

- NC: número de classes
- NA: total de atributos
- DAC: dependencias de relacionamento entre classes
- Análise semântica: Duplicidade e hierarquia

Saída

- Relatório em Markdown com métricas e feedback

1. Métricas utilizadas

Métrica	Descrição
NC — Number of Classes	Contagem total de classes declaradas no diagrama.
NA — Number of Attributes	Total de atributos do diagrama incluindo os injetados pelos relacionamentos normalizado
DAC — Data Abstraction Coupling	Número de atributos cujo tipo é outra classe do diagrama. Indica grau de dependência e entre classes.

2. NA e DAC por classe

Classe	NA	DAC	DAC/NA	Atributos complexos
Cliente	8	3	38%	enderecos (List), carrinho (CarrinhoDeCompras), pedidos (List)
Endereco	6	0	0%	nenhum
Produto	6	1	17%	categoria (Categoria)
Categoria	3	0	0%	nenhum
CarrinhoDeCompras	3	1	33%	itens (List)
ItemCarrinho	4	1	25%	produto (Produto)
Pedido	7	3	43%	itens (List), pagamento (Pagamento), entrega (Entrega)
ItemPedido	4	1	25%	produto (Produto)

Linkando Classes & Requisitos

O que é esta etapa?

- Pós-processamento e Auditoria Automatizada: Esta não é uma etapa de criação do diagrama, mas de validação e congruência. O documenter.py atua após o diagrama UML já ter sido gerado pelas etapas anteriores.

Importância para o Projeto:

- Rastreabilidade: Garante que cada classe no sistema possui uma justificativa de negócio baseada no documento de requisitos e que nenhum requisito crítico foi esquecido no design do diagrama.

Garantia de Qualidade (QA) e Conformidade:

- Identifica automaticamente "Gaps de Implementação" (requisitos não atendidos) e "Classes sem Lastro" (design desnecessário), reduzindo drasticamente o retrabalho e o risco de falha em auditorias de software.

Insumos (Entradas):

1. Diagrama UML (Código Mermaid)
2. Documentação em PDF contento os requisitos

Prompts utilizados para projeto

Etapa 1— Geração de Documentação

Gere o relatório EXATAMENTE no seguinte formato Markdown:

Gere o relatório de rastreabilidade EXATAMENTE no seguinte formato Markdown:

Relatório de Rastreabilidade — Requisitos vs. Diagrama
1. Matriz de Vinculação
Classe UML	Requisito do PDF (ID/Item)	Descrição do Requisito	Justificativa de Design

EXEMPLO:
| {Nome da Classe} | {Ex: Item 4.2} | {Texto curto do PDF} | {Por que essa classe é necessária?} |

Etapa 2. Análise de Conformidade e Gaps
Tipo de Inconsistência	Descrição do Problema	Impacto no Projeto

EXEMPLO:
| Classe sem Lastro | Classe {Nome} não encontrada nos requisitos do PDF. | Baixo (Refatorar ou Remover) |
| Requisito não Atendido | Item {ID} do PDF não possui representação no UML. | CRÍTICO (Funcionalidade Ausente) |

OBRIGADO
PELA ATENÇÃO!